

NOTE PAPER SECFOG

(Giugno 2026) Questo documento è un riassunto destinato a uso personale.

CONTESTO: CLOUD-EDGE COMPUTING

Esistono già tanti lavori su come fare piazzamento ottimale considerando latenza, costo, energia MA **nessuno considera la sicurezza** come criterio di scelta.

- Problemi:
 - I nodi Edge sono fisicamente vulnerabili (possono essere rubati o manomessi)
 - L'infrastruttura è gestita da provider diversi, alcuni potenzialmente non affidabili
 - I dati IoT possono essere sensibili (es. telecamere, dati medici)
- Metodologia proposta:
 - permettere di **dichiarare i requisiti di sicurezza** dell'applicazione
 - permettere di **dichiarare le capacità di sicurezza** dell'infrastruttura
 - considerare il **livello di fiducia** (pesato) verso i vari provider
 - produrre risultati **spiegabili**

SECFOG

Tool prototipato in Problog (pacchetto Python ed estensione probabilistica di Prolog).

Problog permette di:

- esprimere fatti e regole (approccio dichiarativo)
- dichiarare delle probabilità
 - `0.99::firewall(cloud1)` significa "il firewall di cloud1 funziona con probabilità 0.99"
- DUE MODALITA' (**spiegabilità**):
 1. ground mode: genera automaticamente una rappresentazione grafica (un albero AND-OR) che mostra *come* è stato calcolato un certo risultato
 2. explain mode: per ogni query, elenca tutte le prove mutualmente esclusive che portano al risultato.

⇒ Poi il motore di ProbLog propaga automaticamente queste probabilità attraverso tutte le regole logiche, calcolando la probabilità finale che una query sia vera.

Variabili di input:

1. **Security Capabilities** – fornite dagli *infrastructure operators*
Ogni operatore di infrastruttura descrive i propri nodi dichiarando quali capacità di sicurezza ha disponibili e quanto sono efficaci (es: 0.99::firewall(cloud1))
2. **Security Requirements** – forniti dagli *application operators*
Chi gestisce l'applicazione dichiara cosa si aspetta dai nodi su cui vengono deployati i servizi. Ad esempio: "il servizio DataStorage richiede backup + storage cifrato".
3. **Trust Network** – dichiarata da tutti gli stakeholder
Ogni stakeholder può dichiarare quanto si fida degli altri (es: 0.9::trusts(srcOp, aOp)) Queste relazioni formano una rete.

Output: **Security Assessment** – per ogni deployment possibile, SecFog calcola un valore tra 0 e 1 che rappresenta il suo livello di sicurezza complessivo (+ eventuale spiegazione del risultato)

TASSONOMIA della capacità di sicurezza: vocabolario comune per definire i requisiti

#	Categoria	Descrizione	Capacità
1	Virtualizzazione	capacità legate all'isolamento software	access_logs(N) authentication(N) host_ids(N) process_isolation(N) permission_model(N) resource_monitoring(N) restore_points(N) user_data_isolation(N)
2	Comunicazione	capacità legate alla rete	certificates(N) firewall(N) iot_data_encryption(N) node_isolation_mechanism(N) network_ids(N) public_key_cryptography(N) wireless_security(N)
3	Dati	capacità legate ai dati	backup(N) encrypted_storage(N) obfuscated_storage (N)
4	Fisico	capacità fisiche del nodo	access_control(N) anti_tampering(N)

5	Altro		audit(N)*
---	-------	--	-----------

*lett. "verifica" (dei standard di sicurezza, suppongo)

Definire requisiti

Come si descrive l'infrastruttura: con i fatti

```
node(cloud1, cloudOp1) //esiste un nodo chiamato cloud1, gestito da cloudOp1
0.9999::firewall(cloud1) //il firewall di cloud1 funziona con probabilità 0.99
```

Come si descrivono i requisiti dell'applicazione: con i suoi servizi

```
app(smartfarming, [s1, s2, s3]) // l'app smartfarming consiste di tre servizi: s1, s2 e s3
```

Per ogni servizio si dichiarano i requisiti usando il vocabolario della tassonomia. Si possono anche definire **policy personalizzate**, componendo capacità base

```
secureStorage(N) :- // un nodo N offre secureStorage se
    backup(N), //offre backup
    (encrypted_storage(N); obfuscated_storage(N)). // E storage criptato O
    offuscato
```

Poi si usano queste policy per definire i requisiti:

```
securityRequirements(s2, N) :- //il servizio s2 richiede che il nodo N
    secureStorage(N), //offra secureStorage
    resource_monitoring(N). // E resource_monitoring
```

La strategia Generate & Test

//riceve un application operator, un'applicazione e un deployment (inizialmente vuoto), e cerca di completarlo

```
secFog(OpA, A, D) :-
    app(A, L),
    deployment(OpA, L, D).
```

//ricorsiva: prende il primo servizio della lista (C|Cs), cerca un nodo N che soddisfi i suoi requisiti di sicurezza, lo assegna, e poi ripete per il resto della lista.

```
deployment(_, [], []).
deployment(OpA, [C|Cs], [d(C,N,OpN)|D]) :-
    node(N, OpN),
    securityRequirements(C, N),
    deployment(OpA, Cs, D).
```

ProbLog genera automaticamente tutte le combinazioni possibili di assegnamenti nodo-servizio, e per ognuna verifica se i requisiti sono soddisfatti. Quelle che passano il test vengono restituite con il loro score di sicurezza.

Il modello di fiducia di default

```
trusts(X, X). //Ogni stakeholder si fida di sé stesso al 100%
trusts2(A, B) :- trusts(A, B).
trusts2(A, B) :- trusts(A, C), trusts2(C, B). //transitività: se A si fida di B con 0.9, e B si fida di C con 0.8, allora A si fida indirettamente di C
```

Le opinioni lungo un percorso si combinano per **moltiplicazione** (cioè si deteriorano), quelle tra percorsi diversi per **addizione** (migliorano).
OSS: più passi faccio nella catena di fiducia, meno mi fido; ma avere più percorsi verso qualcuno aumenta la fiducia complessiva.

Inoltre, prima di assegnare un servizio a un nodo, si verifica anche che l'application operator si fidi abbastanza dell'operatore di quel nodo.

Limiti del modello di default

1. **Transitività incondizionata**: se A si fida di B e B si fida di C, allora A si fida di C, sempre, senza eccezioni.
2. **Monotonicità**: aggiungere percorsi di fiducia può solo aumentare il livello di fiducia, mai diminuirlo. Nella realtà più percorsi indiretti non sempre migliorano la situazione.

Estensioni algebriche: α SecFog

Un semianello (commutativo) è una struttura algebrica definita da 5 elementi:

$\langle S, \oplus, \otimes, 0, 1 \rangle$

Dove:

- **S** è un insieme di valori
- $\oplus$ è un operatore per combinare valori su percorsi **paralleli** (es. due percorsi diversi verso lo stesso nodo)
 - commutativo e associativo
- $\otimes$ è un operatore per combinare valori in **sequenza** (es. due passi lungo lo stesso percorso)
 - associativo
- **0** è il neutro di $\oplus$
- **1** è il neutro di $\otimes$

OSS: Il modello probabilistico di default di ProbLog è già un semianello $\langle R \cap [0,1], +, \times, 0, 1 \rangle$
... da continuare

NOTE del 20/06/26

Meccanismo generale:

ProbLog calcola la probabilità di una query enumerando tutte le dimostrazioni (proof) mutuamente esclusive che la rendono vera, e sommandone le probabilità (disjoint sum-of-products). Ogni proof è una congiunzione (AND) di letterali positivi/negativi; la probabilità del proof è il prodotto delle probabilità dei letterali.

Esempio 1: Nodo Cloud, senza trust (weatherExample.pl)

Fatti dichiarati:

- anti_tampering=0.99,
- access_control=0.99,
- iot_data_encryption=0.99,
- wireless_security NON dichiarato (=0).

Proof enumerati da ProbLog:

- anti_tampering(cloud) $\wedge$ iot_data_encryption(cloud) $\rightarrow 0.99 \times 0.99 = 0.9801$
- $\neg$ anti_tampering(cloud) $\wedge$ access_control(cloud) $\wedge$ iot_data_encryption(cloud)
 $\rightarrow 0.01 \times 0.99 \times 0.99 = 0.009801$

$\Rightarrow$ Somma = 0.989901

Esempio 2: Nodo Edge, senza trust

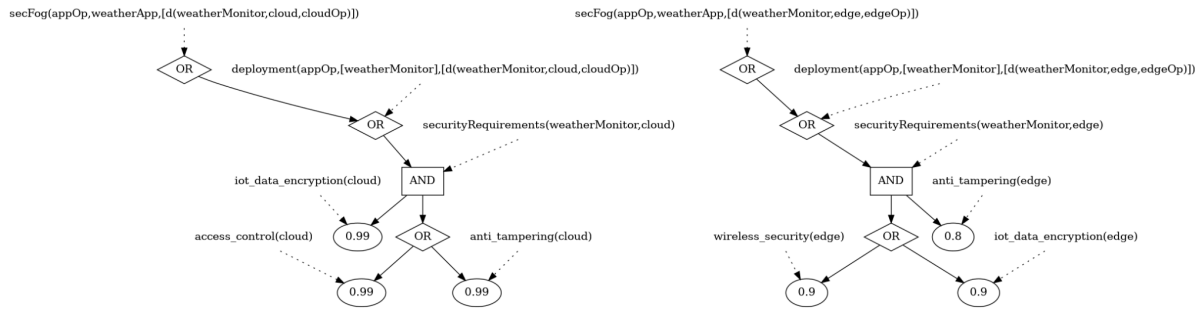
Fatti:

- anti_tampering=0.8,
- wireless_security=0.9,
- iot_data_encryption=0.9,
- access_control NON dichiarato (=0).

Proof:

- anti_tampering(edge) $\wedge$ wireless_security(edge) $\rightarrow 0.8 \times 0.9 = 0.72$
- anti_tampering(edge) $\wedge$ $\neg$ wireless_security(edge) $\wedge$ iot_data_encryption(edge)
 $\rightarrow 0.8 \times 0.1 \times 0.9 = 0.072$

$\Rightarrow$ Somma = 0.792



NOTA: il README ufficiale del repo riporta 0.8415 per questo stesso esempio. Valore verificato a runtime (0.792) DIVERGE dal README. Da capire il perché.

Esempio 3: Nodo Cloud, CON trust (weatherExampleWithTrust.pl)

Il trusts2 è ricorsivo (chiusura transitiva), non uno scalare fisso:

$\text{trusts2}(X,Y) \text{ :- trusts}(X,Y).$

$\text{trusts2}(X,Y) \text{ :- trusts}(X,Z), \text{trusts2}(Z,Y).$

Ogni proof finale = (proof di security capability) AND (una specifica catena di trust da appOp al provider del nodo).

Se esistono più catene possibili, sono trattate come eventi disgiunti (da cui le negazioni $\backslash + \text{trusts}(\dots)$ nei proof, per evitare doppi conteggi).

Esempio proof principale:

$\text{anti_tampering}(\text{cloud}) \wedge \text{iot_data_encryption}(\text{cloud})$

$\wedge \text{trusts}(\text{appOp}, \text{ispOp}) \wedge \text{trusts}(\text{ispOp}, \text{cloudOp})$

$= 0.99 \times 0.99 \times 0.9 \times 0.8 = 0.705672$

Score finale cloud (somma di 12 proof/catene alternative) = 0.76017935

Score finale edge (somma di 4 proof/catene alternative) = 0.755568

Conclusione: Il modello di default SecFog restituisce SEMPRE un valore scalare singolo in $[0,1]$, anche con trust.